

ONNIT

개발 기초 · 서버와 배포

ON · IT · I'M ON IT

BASICS · 01

서버라는 게 뭐냐면.

개발을 시작하면 **서버**, **배포**, **로컬**, **도메인** 같은 말이 계속 나옵니다. 이 말들을 모르면 남이 하는 이야기의 절반을 못 알아듣습니다. 반대로 이것만 잡아두면 나머지는 따라옵니다.

전부 **ONNIT이 실제로 만든 청첩장**을 예로 들어 설명합니다. 추상적인 비유 대신 우리가 방금 한 일을 그대로 씁니다. 읽는 데 십오 분이면 됩니다.

내 컴퓨터 안이나, 바깥이나

이 하나만 구분하면 나머지가 전부 풀립니다. 파일을 더블클릭해서 브라우저에 열면 그건 **내 컴퓨터 안에서만** 벌어지는 일입니다. 화면에 잘 보여도 옆자리 사람은 못 봅니다. 그 파일이 내 하드디스크에만 있기 때문입니다.

남이 보게 하려면 **항상 켜져 있는 다른 컴퓨터에 파일을 올려두고**, 누가 주소를 치면 그 컴퓨터가 파일을 내어주게 해야 합니다. 그 항상 켜져 있는 컴퓨터를 **서버**라고 부릅니다.

나만 파일을 더블클릭

주소가 `file:///Users/...` 로 시작합니다. 이 주소를 카톡으로 보내면 상대방 컴퓨터에서는 열리지 않습니다. 그 사람 컴퓨터에는 그 파일이 없으니까요.

모두 서버에 올림

주소가 `https://...` 로 시작합니다. 링크를 받은 사람은 자기 컴퓨터에 파일이 없어도 열립니다. 서버가 그때그때 보내주기 때문입니다.

서버는 특별한 기계가 아닙니다. **그냥 항상 켜져 있고 인터넷에 연결된 컴퓨터**입니다. 우리 노트북도 서버가 될 수 있습니다. 다만 노트북을 닫으면 사이트가 죽고, 집 인터넷 주소는 통신사가 수시로 바꿔 버립니다. 그래서 남의 서버를 빌려 씁니다.

코드가 사람에게 닿기까지

우리 청첩장이 지금 실제로 이렇게 굴러갑니다. 네 칸으로 되어 있고, 칸마다 하는 일이 다릅니다.



고친 코드가 사람 눈에 닿기까지의 네 칸. 내가 손대는 건 왼쪽 두 칸뿐입니다.

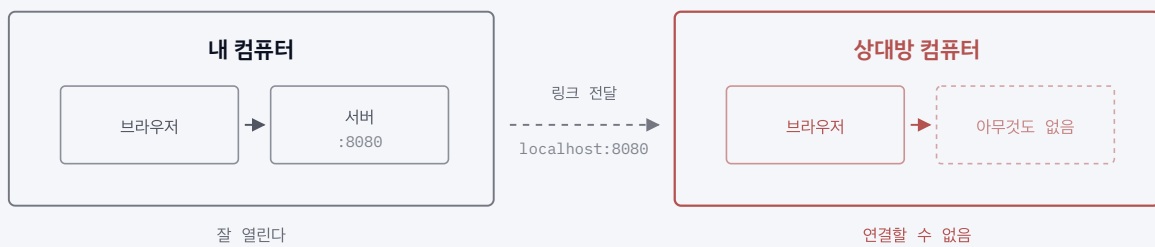
가장 흔한 오해가 여기 있습니다. GitHub에 올렸다고 사이트가 바뀌는 게 아닙니다. GitHub는 **창고**고 배포 서버는 **가게**입니다. 창고에 물건을 넣어도 누군가 진열대에 옮겨야 손님이 봅니다. 우리는 그 옮기는 일을 자동으로 돌게 해줬습니다. `main`에 올리면 일이 분 뒤 사이트가 바뀝니다.

localhost 주소를 보내면 안 되는 이유

개발하다 보면 이런 주소를 자주 보게 됩니다.

```
$ python3 -m http.server 8080
Serving HTTP on 127.0.0.1 port 8080
→ http://localhost:8080
```

이건 **내 컴퓨터를 잠깐 서버로 만든 것**입니다. localhost 는 도메인 이름이 아니라 "**이 컴퓨터 자신**"이라는 뜻의 약속된 말입니다. 그래서 이 주소를 남에게 보내면, 그 사람 브라우저는 **그 사람 컴퓨터**를 찾아갑니다. 거기엔 아무것도 없으니 안 열립니다.



localhost는 "이 컴퓨터"라는 뜻입니다. 같은 주소를 각자 자기 컴퓨터로 해석합니다.

:8080 뒤에 붙는 숫자는 **포트**입니다. 컴퓨터 한 대 안에 여러 프로그램이 동시에 손님을 받을 수 있어서, **창구 번호**를 붙여 구분하는 겁니다. 8080번 창구, 3000번 창구 하는 식입니다.

그래서 "**제 화면에서는 되는데요**"라는 말이 나옵니다. 내 컴퓨터에서 띄운 서버는 나만 볼 수 있습니다. 남에게 보여주려면 반드시 배포를 해야 합니다.

서버가 필요 없는 사이트도 있다

사이트는 크게 두 종류입니다. 이 구분을 알면 왜 어떤 건 파일만 올리면 끝나고 어떤 건 개발이 크게 붙는지 이해됩니다.



정적은 미리 만든 파일을 그대로 보냅니다. 동적은 요청마다 데이터를 읽어 화면을 새로 만듭니다.

우리 청첩장은 **정적**입니다. 누가 열어도 같은 내용이라 파일만 올려두면 됩니다. 그래서 공짜로 배포되고, 접속자가 몰려도 잘 버팁니다.

그런데 **참석 여부와 방명록은 사람마다 다른 내용**입니다. 제대로 하려면 어딘가에 모아 두었다가 신랑 신부에게 보여줘야 합니다. 그러려면 데이터베이스가 필요하고, 그 순간 동적 사이트가 됩니다.

지금 우리 청첩장은 그걸 **각자 브라우저에만** 저장합니다. 내가 쓴 방명록이 내 폰에만 보이고 신랑 신부에게는 안 갑니다. 데모라서 일부러 그렇게 뒀고, 화면 아래에 그렇게 적어 뒀습니다. 실제로 쓰려면 그 부분만 데이터베이스로 바꾸면 됩니다.

도메인과 주소는 다른 것

서버는 원래 `185.199.108.153` 같은 **숫자 주소**를 가집니다. 사람이 외우기 어려우니 **onnit.co.kr** 같은 이름을 사서 그 숫자에 붙입니다. 그 이름이 **도메인**이고, 이름과 숫자를 대조해 주는 전화번호부가 **DNS**입니다.

말	뜻	우리 경우
도메인	사람이 외우는 이름. 1년 단위로 빌린다	onnit.co.kr
DNS	이름을 숫자 주소로 바꿔 주는 전화번호부	도메인 산 곳에서 설정
호스팅	파일을 올려 두고 내어주는 서버를 빌리는 것	GitHub Pages (무료)
HTTPS	주고받는 내용을 암호화. 자물쇠 표시	자동으로 붙는다

도메인을 샀다고 사이트가 생기는 게 아닙니다. **땅 이름만 산 것**이고 건물은 따로 지어야 합니다. 반대로 사이트가 있어도 도메인을 안 붙이면 `lian220.github.io/...` 같은 빌려 쓰는 주소로 다닙니다. 지금 우리 청첩장이 그 상태입니다.

우리 프로젝트에 그대로 넣어 보면

같은 파일이 세 군데에 따로 있습니다. **이 셋이 항상 같지는 않다**는 게 핵심입니다.

어디	무엇이 있나	언제 바뀌나	누가 보나
내 컴퓨터 <code>~/work/onnit-wedding-invitation</code>	지금 고치고 있는 것. 저장만 하고 아직 안 올린 것도 여기 있다	파일을 저장할 때	나만
GitHub <code>github.com/lian220/...</code>	커밋해서 올린 것만. 안 올린 건 없다	<code>git push</code> 할 때	팀 (공개면 전부)
배포된 사이트 <code>lian220.github.io/...</code>	<code>main</code> 에 올라간 것만	push 후 일이 분 뒤 자동	링크를 아는 누구나

사고가 나는 지점이 정해져 있습니다. 고쳤는데 사이트가 그대로다 → 커밋을 안 했거나, push를 안 했거나, 배포가 아직 안 끝난 것입니다. 셋 중 하나입니다. 순서대로 확인하면 됩니다.

1. 안 올린 게 남아 있나

```
$ git status
```

2. 올린 게 GitHub 까지 갔나

```
$ git log --oneline -3
```

3. 사이트가 실제로 바뀌었나 (일이 분 기다린 뒤)

```
$ curl -s https://lian220.github.io/onnit-wedding-invitation/ | grep 찾을내용
```

이 표만 외워도 됩니다

말	한 줄로
서버	항상 켜져 있고, 주소를 치면 파일을 내어주는 컴퓨터
로컬	내 컴퓨터. 나만 보인다
localhost	"이 컴퓨터"라는 뜻. 남에게 보내면 안 열린다
포트	컴퓨터 한 대 안의 창구 번호. :8080
배포	내 코드를 서버에 올려 남이 볼 수 있게 하는 것
정적	누가 봐도 같은 화면. 파일만 올리면 끝
동적	사람마다 다른 화면. 데이터베이스가 붙는다
도메인	서버 숫자 주소에 붙인 이름표. onnit.co.kr
저장소	코드 창고. 여기 올렸다고 사이트가 바뀌지는 않는다

- 모르는 말이 나오면 "그게 내 컴퓨터 안 이야기인가, 바깥 이야기인가"부터 물어보세요. 절반은 그걸로 풀립니다.
- 사이트가 안 바뀌면 커밋 → push → 배포 순서로 확인하세요. 셋 중 하나에서 멈춰 있습니다.
- localhost 가 들어간 주소는 남에게 보내지 마세요. 절대 안 열립니다.